P4209R0
std::numeric_limits for std::basic_vec

Draft Proposal, 2026-04-28

Author:

Daniel Towner (Intel)

Audience:

SG6, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes a partial specialization of `std::numeric_limits` for `std::simd::basic_vec<T, Abi>`. Each trait member has the same value as the corresponding member of `std::numeric_limits<T>` where each value-returning member returns a `basic_vec<T, Abi>` whose elements are equal to the corresponding scalar limit. The specialization is treated as a description of the type model (range, precision, representability), consistent with how `numeric_limits` is interpreted for scalar `T`.

Table of Contents

§ 1. Revision history

- R0: Initial revision.

§ 2. Motivation

§ 2.1. SIMD-generic numeric code

The SIMD working draft introduces `std::simd::basic_vec<T, Abi>` as an element-wise parallel extension of an element type `T`. A central goal is to allow the same generic numeric code to operate on scalar `T` and on `basic_vec<T, Abi>` interchangeably. Most non-trivial numeric code that aims for that behaviour would require `std::numeric_limits<V>` somewhere. For example:

```
template <class V>
bool nearly_equal(V a, V b) {
    return std::abs(a - b) <=
           std::numeric_limits<V>::epsilon() * std::max(std::abs(a), std::a
```

```
}
```

Today, instantiating that function with a simd type is ill-formed, because no specialization of `numeric_limits` is provided for `basic_vec`. The user must either rewrite the algorithm to refer to `numeric_limits<typename V::value_type>`, or abandon the idea of simd-generic behaviour for this function.

The same gap appears in concept-constrained code that tests members such as `is_iec559`, `is_signed`, or `is_integer`: `basic_vec<float, Abi>` silently fails such constraints even though it morally should satisfy them. Also, in standard library facilities specified in terms of `numeric_limits<T>`, such as `std::midpoint`, `std::lerp`, `std::hypot`, or the `<random>` distributions, they are blocked from any future SIMD-generic generalisation while the trait has no answer for `basic_vec`.

`basic_vec` already provides element-wise arithmetic, comparison, and the standard math functions. It already presents itself as a numeric type. `numeric_limits` is the missing piece. Without it, `basic_vec` is missing some meaningful behaviour, where numeric operations and functions work, but the standard mechanism for querying the range, precision, and representability of this type does not.

§ 2.2. Why users should not broadcast manually

A reviewer might suppose that users should be required to write `basic_vec<T, Abi>(numeric_limits<T>::max())` themselves, and leave `numeric_limits<basic_vec<T, Abi>>` unspecialized, but there are three reasons to do something better:

1. The whole point of `basic_vec<T, Abi>` being a numeric type is that generic numeric code written against an arbitrary `V` should work. Forcing every author of generic code to explicitly handle SIMD types defeats this.
2. `numeric_limits` is the standard library's mechanism for querying the type model of `V`. `basic_vec<T, Abi>` has a well-defined type model where it is the same as `T`, applied element-wise, so this relationship is preserved.
3. The same specialization, written once, automatically extends to future extended types (`std::float16_t`, `std::bfloat16_t`, `std::float128_t`, ...), and to any user-defined element type that `basic_vec` may admit, provided that element type specializes `numeric_limits`.

§ 3. Design

§ 3.1. numeric_limits as a description of the type model

§ 3.2. numeric_limits as a description of the type model

`numeric_limits<basic_vec<T, Abi>>` describes the type model of `basic_vec<T, Abi>` (its range, precision, and representability), which is the type model of `T` applied element-wise. It is not a runtime contract, just as scalar `numeric_limits<T>` is not. For example, `numeric_limits<float>::is_iec559` and `numeric_limits<int>::traps` already report values that need not match runtime behaviour under `-ffast-math`, FTZ/DAZ, or trap-disabling configurations, and there is no obligation for `numeric_limits` applied to `simd` to behave differently.

§ 3.3. Element-wise application

Every member of `numeric_limits` falls into one of two categories:

- **Trait members** — static data members of integral, boolean, or enumeration type that describe properties of the type (`digits`, `is_signed`, `radix`, `is_iec559`, `round_style`, ...).
- **Value-returning members** — static member functions that return a value of `T` (`min()`, `max()`, `lowest()`, `epsilon()`, `infinity()`, `quiet_NaN()`, ...).

For `basic_vec<T, Abi>`:

- Trait members have the same value as the corresponding member of `numeric_limits<T>`. They describe the per-element type model.
- Value-returning members return a `basic_vec<T, Abi>` whose elements are each equal to the corresponding scalar limit, obtained by broadcasting through `basic_vec<T, Abi>`'s broadcast constructor.

The user-selected ABI is preserved, so querying `numeric_limits` for `basic_vec<T, Abi>` returns values of exactly `basic_vec<T, Abi>`.

cv-qualified forms (`numeric_limits<const basic_vec<T, Abi>>` etc.) are handled by the existing partial specializations in `<limits>` per [numeric.limits.general], which inherit from the unqualified specialization. No additional specialization for cv-qualified `basic_vec` is required.

§ 3.4. When the specialization is provided

The specialization is provided only when `numeric_limits<T>` is itself specialized (i.e., `numeric_limits<T>::is_specialized` is `true`). For any other `T`, the primary template applies and reports `is_specialized == false`, matching the standard's behaviour for any unsupported type.

This matters because `basic_vec` is expected to allow user-defined element types in future [P2964R3]. Such types automatically gain a sensible `numeric_limits<basic_vec<T, Abi>>` as soon as they specialize `numeric_limits<T>`, with no further library or standardization work.

§ 3.5. Header and module placement

The specialization is provided by `<simd>`, where `basic_vec` itself is declared. Including `<limits>` alone is not sufficient to make the specialization available, so users must include `<simd>`.

§ 3.6. constexpr

`basic_vec<T, Abi>` provides a `constexpr` broadcast constructor. The value-returning members of the specialization are therefore `constexpr` whenever the corresponding members of `numeric_limits<T>` are.

§ 4. Design exploration

The following design choices were considered.

§ 4.1. A separate trait class (e.g., `std::simd::numeric_limits`)

A parallel trait class specifically for SIMD types could be defined alongside `std::numeric_limits`. This was rejected because generic numeric code is written against `std::numeric_limits<V>`, not against a SIMD-specific facility. A parallel trait does not compose with existing generic code, and creates an obligation to keep the two traits synchronised as `std::numeric_limits` evolves.

§ 4.2. Member-by-member specification in the wording

The proposed wording could enumerate each member of `numeric_limits` individually, specifying its value or returned expression. This was rejected because every future change to `numeric_limits` (additions, deprecations, removals) would require a follow-up paper to keep the SIMD specialization in step. The element-wise rule chosen instead tracks `numeric_limits` automatically.

§ 4.3. Multiplying trait values by the SIMD width

A reader might ask whether members such as `digits`, `min_exponent`, or `max_exponent10` should reflect the aggregate properties of a `basic_vec` rather than its element type. For example, by reporting `digits == numeric_limits<T>::digits * basic_vec<T, Abi>::size()`. This is wrong. A `basic_vec` of four `float`s has 4 independent values with 24-bit precision, not one value with 96-bit precision. Generic code that uses `digits` to size a buffer or compute a tolerance would produce garbage. The element-wise interpretation is the only meaningful one.

§ 4.4. Behavioural trait members

The members `is_iec559`, `traps`, `tinyness_before`, `round_style`, `has_infinity`, `has_quiet_NaN`, `has_signaling_NaN`, and `has_denorm` all carry an implicit suggestion of run-time behaviour. On real SIMD hardware:

- FP exceptions are typically masked, so a SIMD operation does not trap on overflow even when scalar `traps` would suggest it might.
- Some instructions ignore the dynamic rounding mode.
- Denormal flushing (FTZ/DAZ) is common, suppressing the representations that `has_denorm` would suggest exist.

Forwarding these members from `numeric_limits<T>` is correct for the same reason it is correct for scalar `T`. `numeric_limits` is the type model, not the runtime contract. Scalar `numeric_limits<float>` reports the same values regardless of `-ffast-math` or the FTZ/DAZ state of the FP environment, and code that needs to know the runtime contract uses `<cfenv>` or compiler-specific facilities, not `numeric_limits`.

§ 4.5. `basic_mask` is out of scope

The SIMD working draft also provides `basic_mask<T, Abi>`. Its elements are boolean, and its `numeric_limits` story would work with `numeric_limits<bool>` in exactly the same way. However, in our current experience SIMD-generic numeric code is never parameterized over a boolean element type. The trait members of interest in simd-generic code (`digits`, `radix`, `epsilon`, `infinity`, etc.) are of no interest. A specialization for `basic_mask` would therefore be of marginal utility, and is not proposed here.

§ 4.6. Deprecated trait members

`has_denorm_loss` is deprecated in C++23 and `has_denorm` is deprecated in C++26. Because the proposal specifies the specialization element-wise rather than member-by-member, removal of these members from `numeric_limits<T>` propagates to `numeric_limits<basic_vec<T, Abi>>` without further specification. Implementations that enumerate members in code must of course track such removals, as they do elsewhere in the library.

§ 5. Implementation experience

The proposal has been prototyped as a single-header partial specialization of `numeric_limits` against an existing implementation of `basic_vec`. The entire specialization is approximately thirty lines of code (see Appendix - Reference implementation) and required no changes to `basic_vec` itself. The existing `constexpr` broadcast constructor is sufficient. The prototype compiles with C++20 and more recent, and behaves as described above for the standard arithmetic element types.

§ 6. Proposed wording

Wording is relative to the current C++ working draft and the SIMD working draft (cited here as [simd]).

Add a new subclause to [simd] called [simd.limits] which would have something like the following wording:

** Numeric limits [simd.limits]**

The header `<simd>` provides a partial specialization of `std::numeric_limits` for `std::simd::basic_vec<T, Abi>`. The specialization is provided if and only if `std::numeric_limits<T>::is_specialized` is `true`.

Let `S` denote `std::numeric_limits<T>` and `V` denote `std::simd::basic_vec<T, Abi>`.

- For each static data member `m` of `S`, `std::numeric_limits<V>::m` has the same type and value as `S::m`. In particular, `std::numeric_limits<V>::is_specialized` equals `true`.
- For each static member function `f` of `S` whose return type is `T`, `std::numeric_limits<V>` provides a static member function `f` whose return type is `V` and whose returned value is `V(S::f())`. The returned value is a `constexpr` expression if `S::f()` is.

[Note: The specialization describes the type model of `V`, which is the type model of `T` applied element-wise. It does not describe the runtime behaviour of operations on `V`. — *end note]*

[Note: The cv-qualified forms `numeric_limits<const V>`, `numeric_limits<volatile V>`, and `numeric_limits<const volatile V>` are provided by the existing partial specializations in [numeric.limits.general], which inherit from the unqualified specialization above. — *end note]*

§ Appendix - Reference implementation

The following self-contained header provides the proposed specialization. It assumes `std::simd::basic_vec<T, Abi>` is already available and provides a `constexpr` broadcast constructor. The cv-qualified forms are handled automatically by the existing partial specializations in `<limits>`.

```
#include <limits>
#include <simd>

namespace std {

template <class T, class Abi>
  requires std::numeric_limits<T>::is_specialized
struct numeric_limits<std::simd::basic_vec<T, Abi>> {
private:
    using V = std::simd::basic_vec<T, Abi>;
    using S = numeric_limits<T>;
```

```
public:
    static constexpr bool is_specialized = true;

    // Value-returning members: broadcast the scalar limit.
    static constexpr V min()           noexcept { return V(S::min()); }
    static constexpr V max()           noexcept { return V(S::max()); }
    static constexpr V lowest()        noexcept { return V(S::lowest()); }
    static constexpr V epsilon()       noexcept { return V(S::epsilon()); }
    static constexpr V round_error()   noexcept { return V(S::round_error()
    static constexpr V infinity()      noexcept { return V(S::infinity());
    static constexpr V quiet_NaN()     noexcept { return V(S::quiet_NaN())
    static constexpr V signaling_NaN() noexcept { return V(S::signaling_NaN
    static constexpr V denorm_min()    noexcept { return V(S::denorm_min()

    // Trait members: forward unchanged from numeric_limits<T>.
    static constexpr int  digits         = S::digits;
    static constexpr int  digits10       = S::digits10;
    static constexpr int  max_digits10   = S::max_digits10;
    static constexpr bool is_signed      = S::is_signed;
    static constexpr bool is_integer     = S::is_integer;
    static constexpr bool is_exact       = S::is_exact;
    static constexpr int  radix          = S::radix;
    static constexpr int  min_exponent   = S::min_exponent;
    static constexpr int  min_exponent10 = S::min_exponent10;
    static constexpr int  max_exponent   = S::max_exponent;
    static constexpr int  max_exponent10 = S::max_exponent10;

    static constexpr bool has_infinity      = S::has_infinity;
    static constexpr bool has_quiet_NaN     = S::has_quiet_NaN;
    static constexpr bool has_signaling_NaN = S::has_signaling_NaN;

    static constexpr bool is_iec559  = S::is_iec559;
    static constexpr bool is_bounded = S::is_bounded;
    static constexpr bool is_modulo  = S::is_modulo;

    static constexpr bool             traps          = S::traps;
    static constexpr bool             tinyness_before = S::tinyness_before
    static constexpr float_round_style round_style    = S::round_style;
};

} // namespace std
```